

Mu2e Readout Controller

Packet Protocol



Fermilab, United States Department of Energy

Maintained by Detector ROC Developers and Managed by Trigger & DAQ Team:

Ryan Rivera, Eric Flumerfelt, Rick Kwarcianny, Ron Rechenmacher

Mu2e-doc-4914

Last Update: 21 February 2026

Refer also to the Command Fan-Out (CFO) User Guide at Mu2e-doc-5619.

Refer also to the Data Transfer Controller (DTC) User Guide at Mu2e-doc-4907.

Document Change Log

DocDB Version	Date	Pages	Editor	Description of Changes
V30	06 Feb 2020	All	rrivera	Added this Change Log. Added dedicated On-spill Flag in Heartbeat packet, which makes event mode 32-bits and byte aligned; Added On-spill Flag in ROC Data Header packet, which makes remaining event mode 7-bits. Also, cleaned up all packet format table depiction.
V33,34	08 Jul 2020	10-11	rrivera	Added documentation on retransmission approach.
V35	20 May 2021	5	rrivera	Clarification on DCS Write and Write Block acknowledgement protocol.
V36	04 Jun 2021	All	rrivera	Clarification and examples added for packet sequence number handling, ROC perspective packet error handling, and DCS reply packet priority.
V37	01 Mar 2023	13-14	carrarin	Update of the definition of the Calo Data Block packets.
V38	28 Jun 2023	17-18	edmonds	Added STM test beam packet definition
V39	31 Jan 2024	All	rrivera	Added improved DCS error handling and clarification on existing error handling.
V41	02 Feb 2024	All	rrivera	Added improved description of sequence number in K28.0 Dx.y packet and marker definitions.
V42	06 Feb 2024	8	rrivera	Fixed typo in Heartbeat Packet subrun bit definition.
V43	07 Feb 2024	6-7, 11-12	rrivera	Changed ROC Status bit definition for Data Headers after discussion with Tracker and Calorimeter ROC FPGA developers (tecchio and carrarin). Improved description of Block Read/Write packet counters.
V44	06 Mar 2024	All	rrivera	Minor marker definition improvements and global clarifications/cleanup.
V45	21 Jun 2024	All	rrivera	Removed Hop Count concept and replaced with reserved bits in packet definitions.
V47	11 Oct 2024	18-19	pgirotti	Added calorimeter debugging data packet format
	28 Jan 2025	11	rrivera	Added TDAQ subsystem ID in preparation for new DTC versions copying ROC subsystem ID into DTC subevent header.
V48	26 Feb 2025	19-20	ehrllich	Updated CRV packet format
V49	07 Mar 2025	11	rrivera	Merged January rrivera unposted modifications with ehrllich version.
V50	06 Aug 2025	4	rrivera	Added negative loopback marker.
V51	12 Jan 2026	11	rrivera	After DAQ Ops meeting, changed definition of ROC Header DTC ID to be exact replica of Subevent Header Data Source DTC ID.
V53	21 Feb 2026	9-10	rrivera	Update Event Mode definition to better define subsystem usage and bits.

1. Introduction

This document defines the data packet protocol used to exchange data between the Mu2e detector subsystem Readout Controllers (ROC)s and the Trigger and Data Acquisition (TDAQ) firmware and software. The Data Transfer Controller (DTC) modules are the immediate interfacing hardware to the ROCs. All communication between the TDAQ software framework and the ROCs must go through the DTCs. This implies that the DTC must be aware of the packet format templates, but not necessarily the packet detail. So the packet templates are also represented in the DTC documentation ([DocDB #4097](#)).

2. Core Functionality Data Packets

The core functionality includes data stream packets (Data Header and Data Payload packets) and Detector Control System (DCS) packets (DCS Read and Write packets).

As described in DCS documentation ([DocDB #3965](#)), the DCS is responsible for providing information about the status and health of the Mu2e detector.

As detailed in the DTC documentation ([DocDB #4097](#)), the DTC modules each have two Direct Memory Access (DMA) ports to provide high bandwidth data transfer between the PCI express and the DTC module's onboard memory. DMA channel 0 is reserved for Readout Request, Data Request, Data Header, and Data packets. DMA channel 1 is used for Detector Control System (DCS) Request and Reply packets.

Each packet has a Packet Type value. The Packet Type is a 4-bit field which maps as follows:

0	DCS Request
1	Heartbeat (broadcast)
2	Data Request
3	Prefetch Request
4	DCS Reply
5	Data Header
6	Data Payload
7	DCS Additional Block Write Payload
8	DCS Reply Additional Block Read Payload
9	Reserved
10-15	Reserved for Special Marker Characters (see DTC-to-ROC Robustness section)

Special Marker Characters are defined as follows:

EventStartK	= 18'h21C10,	//K28.0 D16.0
EventStartKn	= 18'h21CEF,	//K28.0 D15.7
Clock40MHzMarkerK	= 18'h21C11,	//K28.0 D17.0
Clock40MHzMarkerKn	= 18'h21CEE,	//K28.0 D14.7
LoopbackMarkerK	= 18'h21C12,	//K28.0 D18.0
LoopbackMarkerKn	= 18'h21CED,	//K28.0 D13.7
LoopbackMarkerReplyK	= 18'h21C9x,	//K28.0 1001_xxxx (x is ROC src)
DiagnosticK	= 18'h21C13,	//K28.0 D19.0
DCSTimeoutK	= 18'h21C14,	//K28.0 D20.0
RetransK	= 18'h21C15,	//K28.0 D21.0
RetransKn	= 18'h21CEA,	//K28.0 D10.7

Inverted marker pairs, above, are indicated with 'n' at the end of the name; these markers must arrive in pairs for robustness of transmission (to avoid missing the marker by missing a single character in SERDES transmission).

Packet Sequence Numbers

Each packet type transmitted on either the data or CFO links has a unique 2-byte header that is generated by the hardware on transmission and stripped from the packet by the hardware on receipt. This header consists of a K28.0 character, and a Dx.y character. The Dx defines the packet type, and the y specifies the packet sequence number. Packet numbers are reset to zero on system reset and increment by one with each successive packet transmitted. Upon reaching a sequence number of 7, the sequence then repeats beginning at zero. Each ROC, DTC, and CFO module must generate sequence numbers on all packets. Each receiving device must check sequence numbers on incoming packets and flag an error if a packet number is not in sequence. For example, a sequence of data packets might have headers with sequence numbers as follows:

K28.0	D5.0	#data header packet
K28.0	D6.1	#data packet 0
K28.0	D6.2	#data packet 1
K28.0	D6.3	#data packet 2
K28.0	D5.4	#data header packet
K28.0	D6.5	#data packet 0
...etc.		

Note that the sequence number increments by one count regardless of the packet type. Each packet generating device has one sequence number generator. It does not have a separate generator for each receiving device.

The Dx.y bit definition can be thought of as follows:

kk_0001_1100_sssT_tttt

In this definition, kk is kchar for the two 8B10B characters, which are always b10 for packets and markers. b0001_1110 (0x1C) is the comma character for packets and markers. sss is the sequence number, which can be a value 0 through 7, always incrementing. T is a the single bit-4 'Type' defined as T=0 for packets, T=1 for markers. tttt is the nibble 4-bit sub-type for a particular packet/marker.

Detector Control System Packets

DCS Request packets are queued for transmission to the ROCs on DMA channel 1. The DCS Reply packets generated by the ROCs in response to the DCS Request packets are then transferred from the DTC to the DAQ server via DMA channel 1. In hardware, DCS packets have priority over Data packets and are expected to be low bandwidth (Markers have highest/real-time priority). This means that the ROC can respond to DCS packets while handling Data requests, and the DTC supports, for example, a DCS Reply between a Data Header and the Data Payload, or in the middle of Data Payload packets.

The DCS Request packet format is specified as follows:

DCS Request Packet

15

0

<i>K28.0</i>			<i>D0.y</i>	
DMA Byte Count High			DMA Byte Count Low	
Valid	Reserved [14:11]	ROC Link ID [10:8]	Packet Type (0x0) [7:4]	Hop Count [3:0]
Block Op Additional Packet Count [15:6]			Reserved [5]	Op Code [4:0]
Op1 Address [15:0]				
Op1 Write Data [15:0] or Block Op Word Count [15:0]				
Op2 Address [15:0] or Block Write Data0 [15:0]				
Op2 Write Data [15:0] or Block Write Data1 [15:0]				
Reserved or Block Write Data2 [15:0]				
<i>CRC high</i>			<i>CRC low</i>	

The Op Code in the DCS Request Packet is defined as follows:

Bit Position	Definition
--------------	------------

2:0	0 := Read, 1:= Write, 2:= Block Read, 3:= Block Write, 4 := Double Read, 5:= Double Write (Double-ops are not supported by Trk/Calo)
3	Request Acknowledgement
4	Increment Address for Block Operation

The DMA byte count and Valid fields are used by the DTC DMA engine. The ROC can ignore these fields.

The 4-bit Hop Count specifies which ROC on a ring the packet is addressed to.

The Packet Type 4-bit field indicates the type of packet being sent. A DCS Request Packet has type 0x0.

The 3-bit ROC Link ID field specifies the link on parent DTC that the target ROC is located. The DTC's six optical links are numbered 0 through 5.

The **Op Code** 5-bits field specifies the DCS request type. The first 3-bits select Read, Write, Block Read, Block Write, Double Read, or Double Write. A **double operation**, in this context, means two Reads or two Writes requested in a single 16-byte packet. Write data can be ignored for Read operations. For Block Read and Block Write, the **Block Op Word Count** is used to indicate the number of 16-bit words associated with the operation (this can be non-zero for Block Write and Block Read requests); the **Block Op Additional Packet Count** is used to indicate the number of Additional Block Payload 16-byte packets associated with the operation (this should be non-zero for Block Write Requests if the Block Op Word Count is greater than 3).

For Write, Double Write, and Block Write, the **Request Acknowledgement** bit indicates if the ROC should respond with an Acknowledgement packet **AFTER** acting on the Write or Block Write request. An Acknowledgement packet is defined as a DCS Reply Packet with Op Code, Block Op Packet Count, Op1 Address/Data, and Op2 Address/Data mirrored back to the DTC. Note that for Block Write acknowledgements, only one DCS Reply Packet should be sent (i.e. do not mirror the DCS Additional Block Write Payload Packets). Also, note that the Acknowledgement packet should be sent **AFTER** completion of the DCS Write actions in the ROC; this facilitates an automatic approach to throttling DCS writes in software during long configuration sequences, and provides ROC write verification.

For Block Read and Block Write, the **Increment Address** option indicates whether or not to increment the address for each word in the operation. For example, if reading or writing in a FIFO, one might not want to increment the address.

DCS Request Additional Block Write Payload Packet

15

0

<i>K28.0</i>	<i>D7.y</i>
Additional Block Write Data0 [15:0]	
Additional Block Write Data1 [15:0]	
Additional Block Write Data2 [15:0]	
Additional Block Write Data3 [15:0]	
Additional Block Write Data4 [15:0]	

Additional Block Write Data5 [15:0]	
Additional Block Write Data6 [15:0]	
Additional Block Write Data7 [15:0]	
CRC high	CRC low

DCS Reply Packet

15

0

<i>K28.0</i>			<i>D4.y</i>	
DMA Byte Count High			DMA Byte Count Low	
Valid	DTC Errors [14:11]	ROC Link ID [10:8]	Packet Type (0x4) [7:4]	Hop Count [3:0]
Block Op Additional Packet Count [15:6]			Status [5:4]	Op Code [3:0]
Op1 Address [15:0]				
Op1 Read Data [15:0] or Block Op Word Count [15:0]				
Op2 Address [15:0] or Block Read Data0 [15:0]				
Op2 Read Data [15:0] or Block Read Data1 [15:0]				
Reserved or Block Read Data2 [15:0]				
<i>CRC high</i>			<i>CRC low</i>	

In the DCS Reply Packet, for Block Read and Block Write, the **Block Op Word Count** is used to indicate the number of 16-bit words associated with the operation (this can be non-zero for Block Write acks and Block Read replies); the **Block Op Additional Packet Count** is used to indicate the number of Additional Block Payload 16-byte packets associated with the operation (this is non-zero for Block Read replies if the Block Op Word Count is greater than 3).

The Op Code in the DCS Reply Packet is defined as follows:

Bit Position	Definition
1:0	0 := Read(s), 1:= Write(s), 2:= Block Read, 3:= Block Write
2	Double Operation (Not implemented by Tracker/Calorimeter)
3	Acknowledgement Reply

There are several 'non-sense' 4-bit combinations of the Op Code field which are overloaded:

Op Code	Definition
0xC	DTC Identified Invalid Software-to-Chip DCS Request (1100 would otherwise be nonsense "ack of double read")
0xE	DTC Identified ROC Timeout (1110 would otherwise be nonsense "ack of double block read")
0xF	Reserved for software use as 'Unknown' (1111 would otherwise be nonsense "ack of double block write")

The DCS Reply Status 2-bit field is defined as follows (these are flags identified at the ROC FPGA):

Bit Position	Definition
0	“DCS Request FIFO Empty” flag indicates that there are more data requests queued when value is low.
1	“I am corrupt” flag indicates the ROC has lost DCS data or the ability to conduct DCS readout has been compromised.

The DTC Error 4-bit field is defined as follows (these are flags identified at the DTC FPGA):

Bit Position	Definition
0	SERDES PLL associated with the ROC has lost lock
1	SERDES associated with the ROC has lost clock-data-recovery lock
2	Invalid packet has been received from ROC
3	Error in DTC handling of this ROC’s DCS requests has occurred

DCS Reply Additional Block Read Payload Packet

15

0

<i>K28.0</i>	<i>D8.y</i>
Additional Block Read Data0 [15:0]	
Additional Block Read Data1 [15:0]	
Additional Block Read Data2 [15:0]	
Additional Block Read Data3 [15:0]	
Additional Block Read Data4 [15:0]	
Additional Block Read Data5 [15:0]	
Additional Block Read Data6 [15:0]	
Additional Block Read Data7 [15:0]	
<i>CRC high</i>	<i>CRC low</i>

Data Stream Packets

Data stream packets are queued for transmission to the ROCs on DMA channel 0. The Data Reply packets generated by the ROCs in response to the Data Request packets are then transferred from the DTC to the DAQ server via DMA channel 0.

Heartbeat Packet (broadcast)

15

0

<i>K28.0</i>			<i>D1.y</i>		
Transfer Byte Count High			Transfer Byte Count Low		
Valid	Resrv'd [14:11]	ROC Link ID [10:8]	Packet Type (0x1) [7:4]		Resrv'd [3:0]
Event Window Tag byte 1			Event Window Tag byte 0		
Event Window Tag byte 3			Event Window Tag byte 2		
Event Window Tag byte 5			Event Window Tag byte 4		
Event Mode Byte 1 (Resrv'd Trk)			Event Mode Byte 0 [7:3]	Pattern Mode [2:1]	Injection Data Source [0]
Event Mode Byte 3 (Resrv'd CRV)			Event Mode Byte 2 (Resrv'd Calo) [7:1]		Calo Laser Injection [0]
Delivery Ring RF-0 Marker TDC [15:8]			TEM [7:6]	STM [5:4]	Subrun Handling [3:1] On-spill Flag [0]
<i>CRC high</i>			<i>CRC low</i>		

The Heartbeat Packet is guaranteed to arrive from DTC-to-ROC at least 850ns before the Event Marker indicating the start of the Event Window.

The Heartbeat Packet contains ROC Readout Request information specific to the Event Window.

During on-spill, the Delivery Ring RF Marker TDC 8-bit field is an unsigned value time prediction (in units of 800MHz 1.25ns periods) of the offset position of the Delivery Ring Marker with respect to the start of the Event Window, which is specified by the rising edge of 40MHz 25ns System Clock and the Event Window Marker. During off-spill, the TDC value will be 0.

The Heartbeat Packet source is the CFO and contains Event Window specific data that varies from one Event Window to the next. Data for each Event Window is stored in the CFO Heartbeat Information Table in CFO memory. This Table may be static during a run or may be configured over the PCIe on a per Super Cycle basis. The Heartbeat Packet contains the Event Window Tag, along with partition information, in the form of the Event Window Mode, needed to configure the ROCs for the NEXT Event Window, which is specified to begin on the next Event Window Marker. This information is used to control event reconstruction/filtering, ROC data taking, and ROC settings that must be synchronized to a specific μ Bunch (synchronous resets, digitization enable/disable, readout enable/disable, zero-suppression enable/disable, calibration signal injection, live gate expansion, etc.).

The high Event Mode bit, for example bit index 7 of a subsystem's mode byte (or bit 1 of a subsystem's 2-bit mode, is considered the active bit. If set, the corresponding subsystem is expected to record data for that Event Window.

Heartbeat packets and Event Mode can also be used to synchronize commands to precise times within a μ Bunch, by specifying the ROC internal timestamp (time offset from the start of the μ Bunch). This would be used, for example, to request an internally managed ROC calibration signal injection at a particular data sample time. In this case, the mode bits in the Heartbeat Request packet indicate that it is a calibration event (and, if supported, it is the responsibility of the ROC to launch the internal injection behavior – the fine grain offset timing within the Event Window of the injection pulse could be managed

by another ROC register, for example. The μ Bunch ROC Link Mode does not have to be the same as the μ Bunch Mode Byte sent by the CFO – μ Bunch Mode may be decoded by the DTC into the μ Bunch ROC Link Mode using a lookup table populated by DMA.

Subrun handling reserves three bits in the Heartbeat Packet. One approach would be to designate one bit to indicate current subrun is odd/even, and another bit designated as a predictive bit to indicate that an Event Window Tag at a fixed future offset will be in subrun odd/even. The intent of the predictive bit would be to give the software processing chain time to prepare for the subrun transition; and the intent of the current subrun bit would be to verify that the current event is landing in the correct subrun (as was indicated by the predictive bit).

Prefetch Request Packet

15

0

<i>K28.0</i>			<i>D3.y</i>	
Transfer Byte Count High			Transfer Byte Count Low	
Valid	Reserved [14:11]	ROC Link ID [10:8]	Packet Type (0x3) [7:4]	Resrv'd [3:0]
Event Window Tag byte 1			Event Window Tag byte 0	
Event Window Tag byte 3			Event Window Tag byte 2	
Event Window Tag byte 5			Event Window Tag byte 4	
Reserved			Reserved	
Reserved			Reserved	
Reserved			Reserved	
<i>CRC high</i>			<i>CRC low</i>	

The Prefetch Request Packet format is similar to the Data Request Packet format, but it does not lead to a response from front-ends. The Prefetch packet is merely informative – giving front-ends an opportunity to prefetch event window data based on the Event Window Tag to lower latency for the corresponding Data Request Packet to (potentially) follow.

Data Request Packet

15

0

<i>K28.0</i>			<i>D2.y</i>		
Transfer Byte Count High			Transfer Byte Count Low		
Valid	Reserved [14:11]	ROC Link ID [10:8]	Packet Type (0x2) [7:4]	Resrv'd [3:0]	
Event Window Tag byte 1			Event Window Tag byte 0		
Event Window Tag byte 3			Event Window Tag byte 2		
Event Window Tag byte 5			Event Window Tag byte 4		
Reserved			Reserved		
Reserved [15:8]			Debug Type [7:4]	Reserved [3:1]	Debug[0]
Reserved [15:11]		Debug Packet Count [10:0]			
<i>CRC high</i>			<i>CRC low</i>		

The Data Request Packet format is similar to the Heartbeat Packet format, but it is not a broadcast packet. The Debug field is a single bit field, with '1' indicating to ROCs that this is a data request for debugging purposes. If the Debug bit is high, then the Debug Packet Count field indicates the number of debug data packets requested from the specified ROC. The Debug Type field indicates the debug packet type desired. See table below.

<i>Debug Type Code</i>	<i>Description</i>
0xF – 0x3	Reserved
0x2	External serial data with initial reset of FIFO
0x1	External serial data
0x0	Special sequence

The Data Header packet format is specified as follows:

Data Header Packet

15

0

<i>K28.0</i>				<i>D5.y</i>	
Data Block Byte Count High				Data Block Byte Count Low	
Valid	DTC Errors [14:11]	ROC Link ID [10:8]		Packet Type (0x5) [7:4]	Resrv'd [3:0]
Subsystem ID [15:13]		Rsrv'd [12:11]	Packet Count [10:0]		
Event Window Tag byte 1				Event Window Tag byte 0	
Event Window Tag byte 3				Event Window Tag byte 2	
Event Window Tag byte 5				Event Window Tag byte 4	
Data Packet Format Version [15:8]				Status [7:0]	
Event Mode [15:11]		Subrun Handling [10:9]	On-spill [8]	DTC ID [7:0]	
<i>CRC high</i>				<i>CRC low</i>	

The 8-bit Data Packet Format Version number is intended to be a persistent mapping to a particular packet type for the duration of the experiment, so that data can be decoded even if packet definitions change in the future.

The Packet Count 11-bit field is a count, 0 to 2046, indicating the number of Data Payload packets to follow the Data Header packet. 2046 (0x7FE) is the maximum number of packets you can actually support, since there's also the 8-byte DMA size header and the max DMA transfer size is 0x8000 bytes.

The Subsystem ID 3-bit field is defined for each detector subsystem (0 = Tracker, 1 = Calorimeter, 2 = CRV, 4 = STM, 5 = ExtMon, 7 = TDAQ).

The DTC ID 8-bit field for the DTC ID (same value as 'DTC Source DTC ID' in the Subevent Header). The ROC ID from the original ring topology has been replaced with a 4-bit ROC Link ID.

Subrun handling reserves two bits. One approach would be to designate one bit to indicate current subrun is odd/even, and the other bit designated as a predictive bit to indicate that an Event Window Tag at a fixed future offset will be in subrun odd/even. The intent of the predictive bit would be to give the software processing chain time to prepare for the subrun transition; and the intent of the current subrun bit would be to verify that the current event is landing in the correct subrun (as was indicated by the predictive bit).

The Data Header Packet Status 8-bit field is defined as follows:

Bit Position	Definition
0	“Event Window has Data” flag indicates detector data present, else No Data for Event Window.
1	“Invalid Event Window Request” flag indicates the ROC did not receive a Heartbeat packet corresponding to this Data Request.
2	“I am corrupt” flag indicates the ROC has lost data or the ability to conduct detector readout has been compromised.
3	“Timeout” flag indicates ROC retrieval of data did not respond before timeout occurred.
4	“Overflow” flag indicates data is good, but not all data could be sent.
7:5	Reserved

The DTC Error 4-bit field is defined as follows (these are flags identified at the DTC FPGA):

Bit Position	Definition
0	SERDES PLL associated with the ROC has lost lock
1	SERDES associated with the ROC has lost clock-data-recovery lock
2	Invalid packet has been received from ROC
3	Error in DTC handling of this ROC’s DATA requests has occurred

The corresponding data packets that follow the Data Header Packet are defined following this template (where the payload is specified for each detector subsystem):

Data Packet

15	0
<i>K28.0</i>	<i>D6.y</i>
Data 1	Data 0
Data 3	Data 2
Data 5	Data 4
Data 7	Data 6
Data 9	Data 8
Data 11	Data 10
Data 13	Data 12
Data 15	Data 14

<i>CRC high</i>	<i>CRC low</i>
-----------------	----------------

DTC-to-ROC Robustness

The goal is to enhance markers to make the protocol resistant to single character errors. We changed EventStart and 40MHz Markers from a single 16B character to two complimentary characters with no separation (comma) between them. This protects against any single character error condition. Having complimentary characters (instead of just repeating the character) adds robustness because the ROC can still recover the timing of the marker, even if only one of the two characters is recovered. If the ROC recovers only one of the two characters of a marker, it should flag this error and generate a DCS packet with the error condition encoded.

Three scenarios at the ROC are possible:

- 1) Receive Marker character 0. Receive Marker character 1. Generate internal marker signal. No error is detected.
- 2) Receive Marker character 0. Receive Any character other than a Marker character 1. May or may not generate disparity or 8b-10b decode errors. Generate internal marker signal. Count error and make error counter available by DCS request.
- 3) Receive Any character other than a Marker character 0. May or may not generate disparity or 8b-10b decode errors. Receive Marker character 1. Generate internal marker signal. Count error and make error counter available by DCS request.

Marker character pairs examples are as follows:

Event Marker	0x1C10-0x1CEF
40MHz Marker	0x1C11-0x1CEE
Retransmission Marker	0x1C15-0x1CEA

Note: the DelayMeasure (0x1C12) and DCSTimeout (0x1C14) markers both remain just one 16-bit character.

The remaining possible packet errors for DTC-to-ROC transmission, from the ROC perspective, are missing packets and corrupt packets. The ROC can realize a packet is missing by inspecting the packet sequence number; when it does not increment by one for successive packets it can be inferred that a packet was dropped. Corrupt packets can be identified, by a ROC, with a CRC error or unexpected packet contents.

When the ROC detects a Packet Error, the ROC shall:

- 1) Discard the bad packet and all subsequent associated data packets, and abort any operations started by the packet.

2) Flag and count the error (separate resettable counters for missing packets vs corrupt packets would be valuable for debugging, but optional).

There is no retransmission supported by the DTC. It is imagined that, for DCS operations, missing read responses and write acknowledgements will trigger mitigation activity at the software level after a timeout; and for Data operations, missing data responses will trigger error bits and counts in the DTC header packets for subevents and events.

Note that incoming and outgoing packets sequence numbers are completely uncorrelated.

Please refer to the TDAQ Error Handling and Accounting Plans [docdb 22861](#) for additional error handling and scenario considerations.

Note that markers and comma characters can interrupt data transmission from the DTC at any time. For example, this scenario, must be handled correctly by a ROC (i.e., pre_txdata simulation: an Event Marker, a complementary Event Marker, a data word, a comma character, followed by the remaining data words):



ROC-to-DTC Robustness

The goal is to provide a re-transmission mechanism of ROC data packets if an error is detected at the DTC. Packet Error is defined as the following: any packet with a CRC error or a bad sequence number.

When the DTC detects a Packet Error, the DTC shall:

1) Discard the bad packet and all subsequent sequential data packets, until a replacement packet for the bad packet is received, 7 subsequent out-of-sequence packets are received, or a timeout is reached.

2) Flag and count the error.

3) Generate a re-transmit marker containing the sequence number of the last good data packet received. The Retransmit Marker from the DTC is 0x1C15 and 0x1CEA, followed by 16-bit data word with sequence number repeated in each of the four nibbles (e.g. for sequence 3 the 16-bit data word would be 0x3333).

4) If the DTC receives a successful retransmission response from the ROC (the DTC will already have discarded out-of-sequence packets), the DTC propagates the retransmitted in-sequence packets from the ROC. If the DTC receives 7 subsequent out-of-sequence packets from the ROC (before receiving a retransmission packet) or the timeout period passes, then the DTC can assume the ROC retransmission failed and the DTC should reset the next expected sequence number so that communication can continue. A failed retransmission request should be counted at the DTC, and the counter made available to DTC read requests. Before retransmission, the ROC should insert at least 8 comma characters – the DTC uses this as an assertion that the retransmission request was received.

For example, the DTC may receive packet sequence 1,2,4 (which triggers retransmit request 0x2222), then the DTC may receive 5, 3, 4, 5 and this behavior should be acceptable ROC behavior (because

there may be some lag for the ROC to interpret the retransmission request and handle it). In this example, the ROC interpreted the retransmit request after sending 5, which triggers resending 3, 4, 5.

Note that incoming and outgoing packets sequence numbers are completely uncorrelated.

Please refer to the TDAQ Error Handling and Accounting Plans [docdb 22861](#) for additional error handling and scenario considerations.

3. Detector Subsystem Specific Packet Definitions

This section presents the specification for the data packet payload for the Tracker, Calorimeter, and Cosmic Ray Veto subsystems. An additional packet specification is provided for “debug” packets that will be used to test the DAQ system. It is assumed that error-checking is handled separately by the hardware.

Tracker

Data Packet

15	0
K28.0	D6.y
Straw index (16 bits)	
First TDC value (16 bits)	
Second TDC value (16 bits)	
Second ToT value (8 bits)	First ToT value (8 bits)
ADC Sample 1...	ADC Sample 0 (12 bits)
ADC Sample 2...	ADC Sample 1 cont
ADC Sample 3	ADC Sample 2 cont
ADC Sample 5...	ADC Sample 4
CRC high	CRC low

15	0
K28.0	D6.y
ADC Sample 6...	ADC Sample 5 cont...
ADC Sample 7	ADC Sample 6 cont...
ADC Sample 9...	ADC Sample 8
ADC Sample 10...	ADC Sample 9 cont
ADC Sample 11	ADC Sample 10 cont
ADC Sample 13...	ADC Sample 12
ADC Sample 14...	ADC Sample 13 cont
Preprocessing Flags (8 bits)	Reserved [7:4] ADC Sample 14 cont

<i>CRC high</i>	<i>CRC low</i>
-----------------	----------------

The straw index (0 – 23,039) is stored using 16 bits.

The TDC (time-to-digital converter) values, corresponding to the pulse times at the two ends of the straw are each stored using 24 bits.

The ToT (time-over-threshold) values for the two ends of the straw are each stored using 8 bits.

The ADC samples require 12 bits each. For each hit, a fixed number of samples (15) is read out so we pack the samples tightly in order to fit the payload into two data per hit.

One byte is set aside for preprocessing flags. These are filled in by the DTC firmware.

Calorimeter

Each ROC has 20 readout channels, and each readout channel can have multiple hits during a single event window. The calorimeter data payload begins with the Event Header packet, continues with a variable number of Calo Hit Data Packets and ends with an Event Footer packet.

Calo Hit Data Packet

Each hit is readout as a variable length sequence of data packets formatted as follows:

15

Detector Type (3 bits)		Board ID (8 bits)		Channel Number (5 bits)	
Reserved for DIRAC A (16 bits)					
Reserved for DIRAC B (16 bits)					
Sample 0 (12 bits)				Sample 1 (4 higher bits)	
Sample 1 (8 lower bits)			Sample 3 (8 higher bits)		
Sample 3 (4 lower bits)		Sample 4 (12 bits)			
...				...	
...			...		
Sample n-1 (End)		Last Sample Marker (Start)			
Last Sample Marker (End)				Error Flag (Start)	
Error Flags (End)			Time (Start) High Bits		
Time (End) High Bits		Time (Start) Low Bits			
Time (End) Low Bits				Index of Max Sample (Start)	
Index of Max Sample (End)			N of Samples (Start)		
N of Samples (End)		... filled with zeroes up to end of the current packet ...			

0

The first 48 bits of each hit are reserved for the hit header. In particular, the fields are described as follows:

3 bits - Detector Type: subdetector type, CALO = 0, CAPHRI = 1, TRAD = 2, LASER = 3

8 bits - Board ID: unique board identifier from 0 to 255

5 bits – Channel Number: channel identifier from 0 to 19

32 bits – Reserved for future purposes

Immediately following are the digitized samples that form the hit. **Each hit has a variable number of waveform samples** stored contiguously using as many data packets as necessary. Since the packets are formed by 16 bits words, 4 samples fit into 3 words as described in the table. The samples continue until the **Last Sample Marker** (0xFFFF, that corresponds to an unphysical value) is found. The hit ends after 5 words of 12 bits, that are filled with the hit information reconstructed online:

12 bits - Error Flag: a flag that contains info on the quality of the hit

24 bits - Time: the number of clock cycles from the start of the Event Window to the first sample of the hit

12 bits – index of Max Sample: the position inside the hit of the absolute maximum of the waveform

12 bits – Number of Samples: the number of clock cycle from the start to the end of a hit

Following the Number of samples word, if there is any space remaining in the current packet then it is padded with zeroes.

Calo Hit Debugging Mode Data Packet

Alternative Calo Hit data packet “debugADCsv01” used to collect ADC data during the debug phase:

15		0	
0xAAA (12 bits)		Board ID (4 higher bits)	
Board ID (8 lower bits)		Channel ID (8 higher bits)	
Channel ID (4 lower bits)	In Payload Event Window Tag (12 bits)		
Sample 0 (12 bits)		Sample 1 (4 higher bits)	
Sample 1 (8 lower bits)		Sample 3 (8 higher bits)	
Sample 3 (4 lower bits)	Sample 4 (12 bits)		
...		...	
...		...	
Sample n-1 (End)	Last Sample Marker (Start)		
Last Sample Marker (End)		Error Flag (Start)	

Error Flags (End)		Time (Start) High Bits	
Time (End) High Bits	Time (Start) Low Bits		
Time (End) Low Bits		Index of Max Sample (Start)	
Index of Max Sample (End)		N of Samples (Start)	
N of Samples (End)	... filled with zeroes up to end of the current packet ...		

Where all the words are 12 bits. Below instead is listed the “debugCountersV01” calorimeter data format:

Sequential Counter 0 (16 higher bits)
Sequential Counter 0 (16 lower bits)
Sequential Counter 1 (16 higher bits)
Sequential Counter 1 (16 lower bits)
...
...
Sequential Counter n (16 higher bits)
Sequential Counter n (16 lower bits)

Calo Footer Packet

After the transmission of all the hits, the event is closed by a final packet containing the information about all the channels that are reconstructed online in the FPGA:

15		0	
K28.0		D6.y	
Detector Type (3 bits)	Board ID (8 bits)	0	Channel Status Flags A (4 bits)
Channel Status Flags C (16 bits)			
Reserved for DIRAC A (16 bits)			
Reserved for DIRAC B (16 bits)			
Reserved for DIRAC C (16 bits)			
Reserved for DIRAC D (16 bits)			
Reserved for DIRAC E (16 bits)			
Reserved for DIRAC F (16 bits)			
CRC high		CRC low	

The detailed content of the packet has still to be specified based on the collaboration needs. Following the definition used so far:

3 bits - Detector Type: subdetector type, CALO = 0, CAPHRI = 1, TRAD = 2, LASER = 3

8 bits - Board ID: unique board identifier from 0 to 255

4 bits - Channel Status Flag A: mask used to determine the active board channels in the event from 19 (higher bit) to 16 (lower bit)

16 bits - Channel Status Flag B: mask used to determine the active board channels in the event from 15 (higher bit) to 0 (lower bit)

Cosmic Ray Veto

Unlike the other subsystems, the cosmic ray veto (CRV) is only read out if a pull request is made based on the output of TRK and CAL filters. When a pull request is received, all the hits for the event are read out at once, ROC by ROC.

Each CRV DataBlock contains all the hits read out by a single ROC. The Data Header Packet for a CRV DataBlock is immediately followed by a CRV ROC Status Packet with the following format:

CRV ROC Status Packet

15		0	
K28.0		D6.y	
Controller ID (8 bits)		Packet Type (0x06)	Reserved
Controller Event Word Count (16 bits)			
Reserved		Active FEB Flags Byte 2 (8 bits)	
Active FEB Flags Byte 1 (8 bits)		Active FEB Flags Byte 0 (8 bits)	
Trigger Count (16 bits)			
Micro Bunch Status (for Global Run Data: Event Window Tag 0)			
Event Window Tag 1 (for Global Run Data: Event Window Tag 1)			
Event Window Tag 0 (for Global Run Data: Event Window Tag 2)			
CRC high		CRC low	

The 4-bit Packet Type is set to 0x06.

The Controller ID is allocated 8 bits.

Since not every controller will be reading out 24 FEBs, 24 bits are reserved for flags indicating whether a given FEB channel is connected to the controller.

The Trigger Count is incremented once for each data request packet received and is reset each supercycle.

There is a difference between the default firmware that are used for e.g. the VST and the firmware used for the Global Runs. The default firmware uses the last three words for the microbunch status (1 word) and the EWT (2 words, going from highest word to lowest word). The firmware used for the Global Runs uses the last three words for the EWT (going from the lowest word to the highest word).

CRV Hit Readout Format

In the default firmware, immediately following the CRV ROC Status Packet are as many Data Packets as necessary to contain the hit readouts for all the hits recorded by the ROC. Each hit requires 10 words formatted as follows:

15		0	
ROC ID (5 bits)		ROC Port (5 bits)	
# Samples (4 bits)		FEB Channel (6 bits)	
Unused (4 bits)		Hit Time (12 bits)	
Unused (4 bits)		Waveform sample 0 (12 bits)	
Unused (4 bits)		Waveform sample 1 (12 bits)	
Unused (4 bits)		Waveform sample 2 (12 bits)	
Unused (4 bits)		Waveform sample 3 (12 bits)	
Unused (4 bits)		Waveform sample 4 (12 bits)	
Unused (4 bits)		Waveform sample 5 (12 bits)	
Unused (4 bits)		Waveform sample 6 (12 bits)	
Unused (4 bits)		Waveform sample 7 (12 bits)	

This hit format will be changed soon for the new firmware of the CRV FEB-II.

CRV Global Run Format

In the firmware used for the Global Runs, there will be a global run info packet (8 words) following the above mentioned CRV ROC Status Packet.

15		0	
Word 0 (=0xCAFE)			
EWT Count (16 bits)			
Marker Count (16 bits)			
Last EWT (16 bits)			
CRC (8 bits)		PLL (8 bits)	Unused (3 bits)
			Lock (1bit)
Injection Window (16 bits)			
Injection Time (16 bits)			
Word 7 (=0xBEEF)			

This packet is followed by a number of payload packets (8 words each). The number of payload packets can be determined by the packet count in the Data Header Packet (defined in the previous chapter), where two packets (for the ROC Status Header packet and the Global Run Info packet above) need to be subtracted.

Stopping Target Monitor

The following is the data packet definition used by the STM at the ELBE test beam and will be used to develop otsdaq/artdaq software for the STM VST. We have been reading the binary files of this data in Offline with the source module (Sources/src/FromSTMTestBeamData_module.cc) and the packet definitions in Sources/inc/STMTestBeamHeaders.hh.

The overall structure is, Trigger Header #1, Slice Header #1, ADC Samples ... Slice Header #N, ADC Samples, Trigger Header #2, Slice Header #1, ADC Samples...etc. In the test beam data we run over in Offline, we only have one slice per trigger and each slice represents the unsuppressed waveform.

There will be updates to this packet definition very soon.

Trigger Header

15	0
Fixed header word (1, 16 bits)	
Fixed header word (2, 16 bits)	
Data size (1, 16 bits)	
Data size (2, 16 bits)	
Number of Slices (1, 16 bits)	
Number of Slices (2, 16 bits)	
Trigger Number (1, 16 bits)	
Trigger Number (2, 16 bits)	
Trigger mode (16 bits)	
Trigger time (1, 16 bits)	
Trigger time (2, 16 bits)	
Trigger time (3, 16 bits)	
Trigger time (4, 16 bits)	
Trigger time offset (1, 16 bits)	
Trigger time offset (2, 16 bits)	
Number of dropped packets (16 bits)	
Unix time stamp (1, 16 bits, runs 101014 onwards)	
Unix time stamp (2, 16 bits, runs 101014 onwards)	
Unix time stamp (3, 16 bits, runs 101014 onwards)	
Unix time stamp (4, 16 bits, runs 101014 onwards)	

A few more explanations / definitions:

- The 32-bit fixed header word equals 0xDEADBEEF.
- The 32-bit data size represents the total size of the data in this trigger.
- The 32-bit number of slices gives the number of slices that follow the trigger header.
- The 32-bit trigger number is used as the art::Event number in Offline

- The 16-bit trigger mode is either “internal” or “external”. In Offline we have an enum STMTTestBeamEventInfo::TriggerType defined for this (see DataProducts/inc/STMTTestBeamEventInfo.hh)
- The 32-bit trigger time offset is the offset in clock ticks between the trigger time and the first ADC sample
- The 64-bit unix time stamp was added to the trigger header during the test beam for run 101014 onwards.

Slice Header

15

0

Slice Number (1, 16 bits)
Slice Number (2, 16 bits)
Slice Size (1, 16 bits)
Slice Size (2, 16 bits)
Slice Time (1, 16 bits)
Slice Time (2, 16 bits)
Slice Time (3, 16 bits)
Slice Time (4, 16 bits)

A few more explanations / definitions:

- The 32-bit slice size gives the data size of the ADC values after the slice header (number of ADC samples = slice size / 16 bits)

ADC Samples

After the slice header we have N 16-bit ADC samples, where N is determined from the information in the slice header.

Debug Packets

Debug data and header packets are generated when the external serial data Debug Type is 0x1 or 0x2. The number of data packets generated is specified in the DTC readout request. Debug packets are formatted as follows:

Debug Header Packet

15

0

K28.0			D5.y	
Reserved			Reserved	
Valid	Reserved [14:11]	Subsystem ID [10:8]	Debug Type	ROC Link ID [3:0]
0b00000		Packet Count [10:0]		

Timestamp byte 1				Timestamp byte 0
Timestamp byte 3				Timestamp byte 2
Timestamp byte 5				Timestamp byte 4
0xA (0b1010)	0b00	Y	X	Status
# of filled entries in the buffer before readout				# of empty entries in buffer before readout
<i>CRC high</i>				<i>CRC low</i>

The following table lists the valid values for the Packet Type of a Debug Packet.

<i>Debug Type Code</i>	<i>Description</i>
0x0	Special Sequence
0x1	External Serial Data
0x2	External Serial Data With Initial Reset Of FIFO
0x3 — 0xF	Reserved

The packet count fields indicate the number of debug data packets following the debug header packet.

The first bit (labeled X) of the debug header packet following the status field corresponds to the 1-bit flag dbgSerialFifo_empty and indicates whether the FIFO buffer was empty at the time of the readout.

The second bit (labeled Y) corresponds to the 1-bit flag dbgSerialFifo_full and indicates whether the FIFO buffer was full at the time of the readout.

The two bits following the Y field 0.

The following four bits contain 0xA (0b1010).

The 8-bit field labelled “# of empty entries in buffer before readout” corresponds to the dbgSerialFifo_emptyCnt and contains the number of empty entries in the FIFO queue.

The 8-bit field labeled “# of filled entries in buffer before readout” corresponds to the dbgSerialFifo_fullCnt and contains the number of filled entries in the FIFO queue.

The remainder of the header packet is empty.

Debug Data Packet

15

0

<i>K28.0</i>	<i>D6.y</i>
Debug Data 0 (Info)	
Debug Data 1 (Deserialized Data)	
Debug Data 2 (Info)	
Debug Data 3 (Deserialized Data)	
Debug Data 4 (Info)	
Debug Data 5 (Deserialized Data)	

Debug Data 6 (Info)	
Debug Data 7 (Deserialized Data)	
<i>CRC high</i>	<i>CRC low</i>

The debug data packets are divided into 16-bit blocks. If the debug data packet is in response to the external serial data Debug Type (0x1 or 0x2), then each odd-numbered block (Debug Data 1, 3, 5, 7...) corresponds to a 16-bit deserialized word out of the FIFO buffer. The even-numbered data blocks (Debug Data 0, 2, 4, 6...) contain additional metadata formatted as follows:

16-Bit Info Block Format

15					0
0b00	Y	X	0b0	FIFO Count (11 bits)	

Where the FIFO Count is the number of entries in the FIFO buffer and X and Y again correspond to the 1-bit flags dbgSerialFifo_empty and dbgSerialFifo_full, respectively. If the dbgSerialFifo_empty flag is set in a 16-bit info block then the contents of the next 16-bit data block will be identical to the contents of the previous 16-bit data block.

Additional data packets could be defined in the future as needed.